System Architecture & Technical Specifications

Real Estate CRM & Outbound AI WhatsApp Agent Platform

Prepared For: Internal Review & Product Strategy
Date: June 2026
System Version: 3.0 (Production Ready)
Stack Coverage: React (SPA), Node.js (Prisma), Python (LangGraph, pgvector), Twilio

1. System Overview

The application is a state-of-the-art, multi-tenant Real Estate CRM integrated with an autonomous, WhatsApp-enabled conversational AI agent. The stack is split into three decoupled services to ensure scaling independence and isolation of concerns:

- **Frontend (React + Vite + Tailwind)**: A dashboard environment allowing real estate brokers to upload brochures, register leads, launch campaigns, and monitor live chat.
- **Backend Service (Node.js + Express + Prisma)**: Serves as the primary CRM service. Manages authentication, database schemas, Twilio webhooks, and campaign schedules.
- **AI Agent Service (Python + FastAPI + LangGraph)**: Stateful Agent loop using LangGraph that searches matching projects via PostgreSQL pgvector cosine similarity, processes criteria, and books site visits.

2. Technology Stack Breakdown

Layer	Technology	Primary Functionality
Frontend	React (Vite)	SPA providing real-time dashboards, campaigns logs, and chat controls.
Styling	Tailwind CSS	Sleek, modern user interface, layouts, and components.
Backend API	Node.js (Express)	REST API host, database controllers, campaign runner schedules.
ORM	Prisma ORM	Manages PostgreSQL database schemas, transactions, and seed scripts.
AI Service	FastAPI (Python)	High-performance asynchronous API wrapping the agent execution loop.
Agent Logic	LangGraph / LangChain	Stateful multi-agent graph architecture with memory checkpointer.
Vector Search	pgvector (Postgres)	Stores, indexes, and retrieves document embedding chunks.
Media CDN	Cloudinary SDK	Asset uploads (brochures, photos) mirrored directly to global CDN links.

3. Complete API Endpoint Catalog

The API ecosystem spans the CRM backend service and the internal AI engine service.

A. CRM Backend Service Routes (/api)

Endpoint	Method	Scope & Description
/auth/register	POST	Create a new business profile. Rate-limited.
/auth/login	POST	Login, return JWT token. Rate-limited.
/auth/refresh	POST	Refresh JWT access token using a refresh token.
/auth/me	GET	Fetch profile data for currently logged-in account.
/auth/me	PUT	Update profile details (availability, business info).
/leads/upload	POST	Multipart upload of Excel/CSV leads to campaign.
/leads	GET	List active leads (paginated, with sorting).
/leads/:id	GET	Fetch detailed lead profile by ID.
/leads/:id/status	PUT	Update lead stage (pending, nurturing, hot, etc).
/campaigns	POST	Create a new outreach campaign draft.
/campaigns	GET	List campaigns for the business.
/campaigns/:id	GET/PUT	Retrieve or update campaign settings.
/campaigns/:id/launch	POST	Trigger opener template send to all leads.
/campaigns/:id/pause	POST	Pause campaign outreach worker.
/campaigns/:id	DELETE	Delete campaign record.
/kb/create	POST	Create a new KB Vector collection.
/kb	GET	List business KB collections.
/kb/:id/upload	POST	Multipart brochure upload. Mirrors to Cloudinary + pgvector.
/kb/:id/url	POST	Scrape a website URL and embed its text content.
/kb/:id/documents	GET	List source files and chunk metadata.
/kb/:id/document/:docId	DELETE	Purge document chunks from vector DB.
/booking/slots	GET	Fetch available booking slots for a target date.
/booking/book	POST	Book an appointment (manual or agent-driven).
/booking/appointments	GET	Retrieve scheduled site visits.
/booking/availability	GET/PUT	Read or update business work hours.
/booking/appointments/:id/status	PUT	Set status (confirmed, cancelled, completed).
/conversations	GET	List active chat rooms (paginated).
/conversations/:id/messages	GET	Fetch message logs for a specific room.
/conversations/:id/takeover	POST	Assign control to human agent (stops AI replies).

/conversations/:id/release	POST	Release back to autonomous AI agent replies.
/conversations/:id/human-message	POST	Deliver manual text to WhatsApp via Twilio.
/analytics/overview	GET	Fetch overview stats (message counters, rates).
/analytics/campaign/:id	GET	Fetch counters for a target campaign.
/analytics/activity	GET	Fetch activity logs.
/content/generate	POST	Call AI service to generate marketing copy (SMS/Email/etc).
/events	GET	SSE endpoint. Feeds live-chat activity to React UI.
/webhook/whatsapp/incoming	POST	Twilio webhook; catches user reply & calls AI agent.
/webhook/whatsapp/status	POST	Twilio webhook; records message status changes.

B. AI Service Internal Routes (Port 8000)

Used internally by backend nodes to query conversational steps or generate content templates.

Endpoint	Method	Input Payload / Return Description
/agent/message	POST	Payload: {thread_id, message, kb_id, campaign_config} Return: {reply, qualification_score, stage, needs_human, brochure_url}
/kb/{kb_id}/file	POST	Payload: File + Description. Returns Cloudinary URL & chunk count.
/kb/{kb_id}/documents	GET	List source filenames and page content details.
/kb/{kb_id}/documents/{source}	DELETE	Delete document chunks from PostgreSQL pgvector.
/content/generate	POST	Payload: {type, brief, tone}. Generates marketing copies.
/health	GET	Simple ping route returning {status: 'ok'}.

4. Server-Sent Events (SSE) Architecture

Real-time event synchronization is crucial to let agents take over conversations instantly. We choose **Server-Sent Events (SSE)** over WebSockets to maintain a lightweight, native, unidirectional push mechanism (Server to Client) with auto-reconnection out of the box.

How It Operates:

1. **Persistent Tunnel**: React requests `GET /api/events` with JWT credentials. Express responds with `Content-Type: text/event-stream` and keeps the socket alive.
2. **Tenant Mapping**: The backend `sseService` groups open response sockets by `businessId`. When a webhook receives a user message or sends an agent reply, it loops through the associated client sockets

and calls `res.write()`.

3. **Audio & Toast Chime**: If the pushed message carries a `hot_lead` trigger or a stage set to `handoff`, the React hook (`useSSE.js`) plays a dual-frequency synthesizer chime utilizing the Web Audio API.

5. Current Pros, Cons, and Architecture Limits

Current Strengths (Pros)	Architectural Trade-offs / Limits (Cons)
• Stateful Memory: LangGraph checkpointer persists intent signals, scores, and history state perfectly across all turns.	• Sync Webhook Listener: The Node webhook awaits FastAPI synchronously
• Media Delivery: Auto-fallback pushes Cloudinary links directly in the message body if media payload fails.	• Sandbox Restriction: In development mode, leads must text a join phrase to
• Strict Capping: 900-char text cap protects Twilio text length limits and avoids mid-text responses.	• LLM Rate Limits: Occasional 429s under heavy stress testing on Groq/Mistr

6. Future Roadmap Suggestions

1. **Asynchronous Task Worker**: Implement a message queuing layer (e.g. BullMQ & Redis). Node webhook immediately answers Twilio 200 OK, queuing the AI response task asynchronously. This completely mitigates Twilio timeout issues.
2. **Production WhatsApp Profile**: Transition to an official WhatsApp Business API profile. This bypasses sandbox constraints, allows native interactive buttons, list options, and removes sandbox opt-in rules.
3. **Hybrid Keyword/Semantic Search**: Integrate BM25 search index on pgvector. This allows exact match scoring on specific strings (like flat size, floor numbers, prices) where semantic embeddings are occasionally imprecise.